Byzantine-Tolerant Belief Aggregation over Merkle-DAGs: A Deterministic Epistemic Layer for Decentralized AI

Fabrizio Salmi

Independent Researcher

March 2026 — Technical Report / Preprint

Abstract. Conflict-free Replicated Data Types (CRDTs) guarantee that distributed nodes converge on the same data, but they do not guarantee that the data is *true*. We present an Epistemic Layer (L3) built above AIMP, a high-performance Merkle-CRDT protocol achieving 1.28M mutations/sec with Ed25519 signing. Designed for multi-agent systems, L3 maps noisy sensor data and LLM inferences into BFT-safe logical graphs. Our contributions are: (1) deterministic epistemic fusion via integer log-odds arithmetic, eliminating IEEE 754 non-reproducibility across architectures; (2) two-pass Markovian trust relaxation that normalizes flow to prevent diamond amplification over acyclic subgraphs, convergent with spectral radius $\rho(A) = 0$; (3) Sybil-resistant reputation via cryptographic Web of Trust delegation with reputation spending; and (4) materialized compaction via grid quantization and holographic routing, guaranteeing strict CRDT deduplication and topological resilience across asynchronous garbage collection epochs. Together, these mechanisms establish *Eventual Epistemic Convergence* (EEC): when the underlying AP data plane converges, the belief plane converges instantaneously and deterministically, without requiring synchronous BFT coordination.

We evaluate L3 with Criterion micro-benchmarks (full belief pipeline in 140 µs for 1,000 claims), scalability tests (trust propagation over 10,000-claim sparse graphs in 1.75 ms), property-based testing of algebraic invariants (7 properties, 256 cases each), exhaustive bounded verification (5 safety properties over 199,902 configurations), and a TLA+ formal specification. We compare quantitatively against Subjective Logic and Dempster-Shafer Theory: L3 is 98–142× faster at aggregating 1,000 evidence items and is the only framework producing bit-identical results across architectures — a prerequisite for Eventual Epistemic Convergence. As a concrete application, we benchmark credential revocation propagation, achieving 40 µs computation per node for 100-node meshes. Combined with L2 gossip (under 1 ms/hop, measured in Paper 1), end-to-end revocation is under 10 ms — compared to ~100 ms for OAuth token introspection or ~1 hour for OCSP stapling. The implementation is open-source (Rust, MIT license).

1. Introduction

Conflict-free Replicated Data Types (CRDTs) [1] solve the fundamental problem of replicated state convergence in partition-tolerant networks. By ensuring that concurrent updates commute, CRDTs guarantee *eventual consistency* without coordination. This property makes them attractive for multi-agent systems where autonomous agents operate under intermittent connectivity.

However, CRDTs are *semantically blind*: they ensure all nodes see the same set of facts, but provide no mechanism to assess whether those facts are *true*, *trustworthy*, or *contradictory*. When agents produce observations from noisy sensors, inferences from local models, and intents to act in the physical world, the system must reason about epistemic quality — not merely data presence.

We address this gap with an Epistemic Layer (L3) that sits above a high-performance Merkle-CRDT transport layer (AIMP v0.1.0 [2]). L3 interprets opaque CRDT payloads as typed *knowledge claims*

with provenance, confidence, and causal structure. It classifies claims into *accepted*, *rejected*, or *uncertain* beliefs through reputation-weighted Bayesian aggregation — all in deterministic integer arithmetic.

The key insight is *architectural orthogonality*: L3 never blocks L2. The CRDT merges unconditionally at full speed (1.28M ops/sec); the epistemic layer processes the merged state asynchronously. This preserves the availability guarantees of the AP data plane while adding a convergent belief plane on top. We call this property *Eventual Epistemic Convergence* (EEC): L3 does not impose synchronous BFT consensus (which would require blocking writes and violate the AP guarantee). Instead, L3 provides a *BFT-deterministic validation function*: any two nodes with the same L2 state are guaranteed to produce bit-identical L3 beliefs. When L2 eventually converges (as CRDTs guarantee), L3 converges instantly and deterministically. During network partitions, nodes in different partitions may temporarily hold different beliefs — but they can never hold *inconsistent* beliefs given the same data.

1.1. Contributions

1. **Deterministic Epistemic Fusion via Integer Log-Odds.** We replace floating-point confidence with integer log-odds ($i32$, milli-log-odds scale). Epistemic fusion of independent evidence reduces to pure addition; aggregation becomes summation. Zero underflow, zero architecture-dependent rounding, 100% deterministic.

2. **Two-Pass Markovian Trust Relaxation.** We separate positive propagation (Supports/DerivedFrom edges) from contradiction subtraction. Pass 1 uses *Markovian flow normalization*: trust is divided across outgoing edges proportionally to their strength (stochastic matrix), preventing diamond amplification. Cyclic edges are zeroed via sorted DFS. Pass 2 applies contradictions *simultaneously* using frozen values (out-of-place update). No oscillation.

3. **Sybil-Resistant Reputation.** New nodes start with reputation 0. Voting weight requires cryptographic delegation from an established anchor node (Web of Trust). Delegated reputation is capped by the delegator's own score. Equivocation-slashed nodes cannot delegate.

4. **Materialized Compaction via Grid Quantization and Holographic Routing.** To survive L2 garbage collection without topological severing, orphaned epistemic edges gracefully degrade to resolve via Semantic Fingerprints, dynamically reconnecting to materialized Summaries. To prevent confidence double-counting during split-brain network merges, Semantic Reducers are strictly quantized to temporal epoch grids, guaranteeing BFT-deterministic deduplication of asynchronous summaries.

5. **Quantitative Evaluation.** We provide Criterion micro-benchmarks, scalability analysis, property-based tests (proptest), exhaustive bounded model checking (199,902 configurations), a TLA+ formal specification, SOTA comparison against Subjective Logic and Dempster-Shafer, and a credential revocation application benchmark with network impairment simulation.

2. Background and Related Work

2.1. CRDTs and Eventual Consistency

CRDTs, formalized by Shapiro et al. [1], provide mathematically guaranteed convergence through commutative, associative, and idempotent merge operations. Merkle-DAG CRDTs, explored by Kleppmann [3], extend this with cryptographic integrity: each node in the DAG is content-addressed, enabling efficient delta-sync and tamper detection.

AIMP v0.1.0 [2] implements a Merkle-DAG CRDT with Ed25519 per-mutation signing, Noise Protocol XX transport, and BFT quorum consensus. It achieves 1.28M mutations/sec in batch mode and sub-millisecond convergence. The present work builds upon AIMP's L1/L2 stack.

2.2. Byzantine Fault Tolerance in Distributed AI

Castro and Liskov's PBFT [4] established practical Byzantine consensus. Tendermint [5] and Casper [6] introduced *equivocation slashing* — cryptographic proof that a node signed contradictory messages. AIMP v0.1.0 adapts this pattern to CRDTs, expelling Byzantine nodes while preserving unconditional set-union merge.

The present work extends BFT from the data plane to the *belief plane*: not only detecting Byzantine data injection, but limiting its epistemic influence through reputation-weighted trust propagation.

2.3. Belief Aggregation and Trust Networks

Bayesian belief aggregation in multi-agent systems has been studied extensively [7]. The challenge in decentralized settings is *floating-point non-determinism*: IEEE 754 arithmetic produces architecture-dependent rounding, breaking consensus. We solve this with log-odds integer arithmetic, following the insight that Bayesian update in log-odds space reduces to addition [8].

Trust propagation on graphs relates to PageRank [9] and Markov Random Fields. Our two-pass approach avoids the oscillation problems of signed-weight relaxation by separating positive and negative influence.

2.4. Subjective Logic

Jøsang's Subjective Logic [10] represents beliefs as (b, d, u, a) tuples (belief, disbelief, uncertainty, base rate) and provides fusion operators for combining opinions. While mathematically elegant, its reliance on IEEE 754 floating-point arithmetic makes it unsuitable for BFT consensus where all nodes must produce identical results. We provide a quantitative comparison in Section 11.

2.5. Dempster-Shafer Theory

Dempster-Shafer theory [11] assigns belief mass over a power set of hypotheses. Dempster's rule of combination is known to produce pathological results (the Zadeh paradox) when evidence sources strongly conflict. Additionally, the $O(2^{|\Theta|})$ complexity of the power set limits scalability. Our approach avoids both issues through pairwise integer aggregation.

2.6. Reputation Systems

EigenTrust [12] computes global reputation via PageRank-like iteration on a P2P network. Unlike EigenTrust, L3 does not rely on float-based iterative convergence: reputation is *delegated* (explicit, deterministic) rather than *computed* (iterative, float-dependent). New nodes start at reputation 0 (not a uniform prior), and closed cliques of mutually-endorsing nodes cannot inflate their own reputation because delegation *costs* the delegator (reputation spending, Section 6.2).

Oracle networks. Chainlink's Decentralized Oracle Networks (DONs) solve a related problem — aggregating off-chain data for on-chain consensus — but require on-chain contract execution (gas costs, seconds-to-minutes latency). L3 achieves epistemic convergence in under 10 ms entirely off-chain, integrated natively into the node's local state. The two approaches are complementary: L3 could serve as an off-chain pre-filter that feeds aggregated beliefs into an on-chain oracle.

3. System Architecture

The AIMP stack is organized in three layers:

Layer	Responsibility	Crate
L3: Epistemic	Belief, trust, contradiction, compaction	`aimp-epistemic`
L2: State	CRDT merge, epoch GC, delta-sync	`aimp-core`
L1: Transport	Noise XX, gossip, Ed25519 identity	`aimp-core`

Table 1: AIMP layered architecture.

Invariant: L3 never blocks L2. The CRDT merges at full speed regardless of cognitive processing. L3 deserializes opaque L2 payloads on the consumer side only.

Backpressure. If L2 produces mutations faster than L3 can process the belief graph (e.g., 1.28M ops/sec on L2 vs. ~7K claims/sec on L3 for dense graphs), L3 may fall behind. The design handles this via *state-based snapshot processing*: when the L3 queue exceeds a configurable threshold, L3 discards intermediate computations and recomputes beliefs from the latest stabilized L2 state. This trades incremental precision

for bounded memory and bounded lag, preventing out-of-memory conditions on resource-constrained nodes.

4. Deterministic Epistemic Fusion

4.1. Motivation

Standard probability representation ($p \in [0, 1]$) requires multiplication for Bayesian update: $P(\text{posterior}) = P(\text{prior}) \times \frac{P(\text{likelihood})}{P(\text{evidence})}$. Recursive multiplication of fixed-point integers causes *underflow to zero* — a fundamental problem for any integer-arithmetic Bayesian system.

4.2. Log-Odds Representation

We represent confidence as *log-odds* scaled by 1000 (milli-log-odds), stored as `i32`:

$$\text{logodds}(p) = 1000 \times \ln\left(\frac{p}{1-p}\right)$$

Log-Odds (i32)	Probability
−6907	~0.1%
−2197	~10%
0	50%
+2197	~90%
+6907	~99.9%

Table 2: Log-odds integer mapping to probability.

Bayesian update becomes addition:

$$\text{posterior} = \text{prior} + \text{evidence}$$

Aggregation of N independent sources:

$$\text{combined} = \sum_{i=1}^{N} \text{evidence}_i$$

All operations are saturating `i32` addition — a *Fixed-Point Log-Odds Isomorphism* that maps the multiplicative Bayesian update to additive integer arithmetic. No multiplication chains, no underflow, no architecture-dependent rounding. Conversion to/from probability uses a deterministic lookup table (no `ln`/`exp`).

ZK-readiness. Unlike float-based systems, L3′s exclusive use of bounded integer arithmetic makes the belief engine intrinsically compatible with zero-knowledge proof systems. The entire trust propagation pipeline (saturating addition, integer comparison, DFS over adjacency lists) operates within the finite field arithmetic native to zk-SNARK and STARK circuits. This opens the path to *Zero-Knowledge Epistemology*: a node can prove it computed beliefs correctly without revealing its local subgraph, enabling privacy-preserving belief aggregation in adversarial networks.

Ontological Priors. In the Bayesian framework, every assertion requires a prior. In L3, a claim's initial log-odds is 0 (the Principle of Indifference: 50% probability). However, the Belief Engine pipeline (Section 8) immediately weights this by the author's reputation: $\text{base_trust} = \text{reputation}_A \times \text{confidence}_i$. A claim from a highly reputable node ($\text{rep} = 10000$) enters the graph at full confidence. A claim from an unknown node ($\text{rep} = 0$) enters at zero effective weight, regardless of self-declared confidence. Thus, the author's reputation functions as the Bayesian prior: the network's accumulated trust in the source *is* the prior probability that the claim is true.

4.3. Confidence Intervals

To address the expressiveness gap with Subjective Logic (which has an explicit uncertainty dimension u), we introduce `ConfidenceInterval`: a pair $[\text{lower}, \text{upper}]$ in log-odds space. Cost: 8 bytes ($2 \times$ `i32`).

Aggregation: lower = min(lowers), upper = max(uppers), midpoint = Bayesian sum of midpoints. Width grows when sources disagree, shrinks when they agree (via intersection). All operations are deterministic integer arithmetic.

`ConfidenceInterval` is a *local computation tool*, not a CRDT-safe type. Its `aggregate` function uses min/max range tracking, which is not associative under partial merges. The scalar `LogOdds` remains the sole inter-node replication type. `ConfidenceInterval` is computed locally from converged `LogOdds` values when single-node uncertainty analysis is needed.

Quantization note: The `from_percent` function returns exact milli-log-odds for a given probability (e.g., 90% → 2197). The inverse `to_percent` uses coarser brackets for human readability (e.g., 2197 → "95%"). These functions are convenience utilities; all computation uses the exact integer log-odds value, which is never quantized.

4.4. Echo Chamber Protection

When N nodes relay the same sensor reading, naive aggregation would count it N times. Each claim carries an `evidence_source` field — the BLAKE3 hash of the *original* data source. The aggregator counts only *unique* evidence sources.

Limitation (Sybil): This prevents *network-level* amplification (same data relayed by N nodes) but not *correlated physical sensor failure* (two broken sensors reading the same wrong value).

Limitation (Independence): Log-odds summation assumes conditional independence of evidence sources (the Naive Bayes assumption). When K sensors observe the same correlated phenomenon, the aggregate log-odds grows as $K \times \text{evidence}$ even though the true information content is less than K-fold. This produces pathological hyper-confidence. The `evidence_source` deduplication catches identical data but not correlated data from distinct sources. Mitigation via correlation factors (e.g., mutual-information-weighted discounting or copula-based correction) is future work. Applications must be aware that L3's aggregation is optimistic under correlation.

5. Trust Propagation

5.1. Knowledge Graph

Claims are connected by typed epistemic edges. In AI agent deployments, edges are created via structured output: agents emit JSON-encoded edge declarations (e.g., `{"relation": "supports", "target": "<hash>",`
`"strength": 8000}`). For LLM-based agents, semantic similarity between claim payloads (e.g., cosine similarity of quantized embeddings) can be mapped to edge types: similarity above a threshold produces Supports, below produces Contradicts, and causal derivation produces DerivedFrom. The mapping function is application-specific and may itself be non-deterministic (e.g., LLM-based classification). L3 decouples *semantic intent generation* (which may be probabilistic) from *epistemic graph resolution* (which is strictly BFT-deterministic). Once a node signs an edge cryptographically, that edge becomes an immutable fact in the Merkle-DAG. L3 does not guarantee that the LLM's classification was correct; it guarantees that, given the same set of signed edges, every node in the network reaches the exact same epistemic conclusion.

Relation	Semantics
Supports	Source provides evidence for target
Contradicts	Source disputes target
DerivedFrom	Target was inferred from source
SharedSource	Same evidence origin (potential echo chamber)

Table 3: Epistemic edge relations and semantics.

5.2. Two-Pass Markovian Trust Relaxation

Mixing positive (Supports) and negative (Contradicts) edges in iterative relaxation causes oscillation on non-bipartite graphs. We separate them:

Pass 1 (Markovian Positive Flow). Fixed-point iteration: $\vec{t}_{k+1} = \vec{t}_0 + A\vec{t}_k$, where $\vec{t}_0$ is the base trust vector and A is the weighted adjacency matrix of Supports/DerivedFrom edges. Cyclic edges identified via DFS back-edge detection are excluded (weight = 0).

BFT-safe DFS ordering. The DFS traversal visits nodes in sorted order by `ClaimArenaId`, and outgoing edges from each node are sorted by target ID before traversal. This ensures the DFS spanning tree — and thus the set of identified back-edges — is bit-identical on all nodes regardless of `HashMap` iteration order or gossip message arrival sequence. Without this ordering guarantee, different nodes could identify different back-edges in the same cycle, producing divergent trust values and breaking BFT consensus.

Epistemic cost of cycle breaking. Zeroing back-edges discards the epistemic weight of circular mutual support (e.g., A supports B, B supports C, C supports A). This is a deliberate trade-off: in BFT systems, convergence (liveness) takes priority over representing circular tautologies. An alternative approach — collapsing Strongly Connected Components into single "Super-Claims" before propagation — would preserve the intra-cycle coherence while maintaining acyclicity. This refinement is left to future work.

The adjacency matrix is *stochastic* (Markovian): each source node divides its trust proportionally across outgoing edges, preventing diamond amplification. For each non-cyclic edge e from source i:

$$\text{contribution} = \max(0, \text{trust}_i) \times \frac{\text{strength}_e}{\sum_{e' \in \text{out}(i)} \text{strength}_{e'}} \times \frac{\text{reputation}_A}{10000}$$

This is a *Conservation of Epistemic Mass* law: the total trust flowing out of a node equals the trust flowing in (times decay). A diamond topology ($A \to B \to D$, $A \to C \to D$) cannot amplify A's influence on D beyond A's own reputation, regardless of the number of intermediate paths.

The $\max(0, \cdot)$ clamp ensures rejected claims have zero epistemic authority. The use of reputation_A (not trust_i) prevents type confusion between log-odds and reputation.

Pass 2 (Contradiction Subtraction). Simultaneous $O(E)$ sweep using *frozen* stabilized values from Pass 1 (out-of-place update). All penalties are computed against the frozen snapshot, then applied. This prevents topological evaluation bias: if A contradicts B and B contradicts C, the order of edge iteration cannot affect the result because both penalties reference the frozen Pass 1 values, not the in-progress Pass 2 state. For each Contradicts edge:

$$\text{penalty} = \max(0, \text{trust}_{\text{source}}) \times \frac{\text{strength}_e \times \text{reputation}_A}{10000^2}$$

$$\text{capped_penalty} = \min\big(\text{penalty}, \alpha \times \max\big(0, \text{trust}_{\text{target}}\big)\big)$$

where $\alpha \in [0, 1]$ is the configurable *Contradiction Damping* factor (default: 0.5). This limits a single high-trust node from annihilating another's trust in one operation.

Convergence Claim. After DFS back-edge zeroing, the positive sub-adjacency matrix A is strictly upper triangular with respect to the DFS topological ordering. All eigenvalues of A are 0, hence $\rho(A) = 0 < 1$. The fixed-point iteration $\vec{t}_{k+1} = \vec{t}_0 + A\vec{t}_k$ converges in at most D steps, where D is the maximum depth of the acyclic subgraph. Pass 2 is a single $O(E)$ sweep, requiring no iteration.

Corollary. The Belief Engine produces identical results on all nodes regardless of message arrival order, satisfying BFT determinism.

Status: This claim is verified empirically via exhaustive bounded testing over 199,902 graph configurations (Section 10.2), property-based testing of determinism (Section 11.6), and TLA+ model checking (Section 10.1). A mechanized proof is future work.

5.3. Dynamic Decay

Trust decay per edge is not a protocol constant but the *effective weight* of the edge: $\text{strength}_e \times \text{reputation}_{\text{author}}$. An edge with strength 9500 (95%) barely attenuates; an edge with strength 1000 (10%) heavily attenuates. The network determines its own trust topology.

5.4. Temporal Decay

Claims lose trust weight over time via integer half-life decay. Given a claim with Lamport tick t_c and current Lamport tick t:

$$\text{trust}_{\text{decayed}} = \text{trust}_{\text{base}} \gg \left\lfloor \frac{t - t_c}{\text{half_life}} \right\rfloor$$

This is a right bit-shift — zero-cost, deterministic, bounded (30 shifts → effectively 0). Old claims naturally fade without active GC.

BFT safety: The tick t is a *Lamport timestamp* (L2 epoch counter), not wall-clock time. Wall-clock time is unusable in BFT settings because Byzantine nodes can falsify NTP-based timestamps. Lamport ticks are monotonically assigned by the CRDT layer and are unforgeable (tied to the Merkle-DAG causal structure).

5.5. Dynamic Contradiction Damping

Rather than a static damping cap α, we compute α dynamically based on the support/contradiction ratio for each target claim:

$$\alpha = \min\left(\frac{\text{contradictions}}{\text{supports} + \text{contradictions} + 1}, \alpha_{\max}\right)$$

The +1 in the denominator is Laplace smoothing. The ratio uses *trust-weighted* support and contradiction counts (not raw edge counts) to prevent Sybil manipulation: an attacker flooding zero-reputation Contradicts edges cannot inflate α because zero-trust edges contribute zero weight to the denominator. Additionally, the total accumulated penalty across all contradiction edges is globally capped at $\alpha_{\max}$% of the target's stabilized trust, preventing "death by a thousand cuts" where K individually-capped edges collectively exceed the limit.

6. Sybil Defense

6.1. The Attack

An adversary generates K fresh Ed25519 keys. If default reputation is $R_0 > 0$, total fake voting weight is $K \times R_0$ — unbounded.

6.2. Web of Trust

New nodes start with reputation 0. To gain voting weight, a node must be *delegated* by an existing trusted node:

$$\text{delegated_rep} = \min(\text{requested_rep}, \text{delegator_rep})$$

Delegation rules:

- Capped by delegator's own reputation (cannot grant more than you have)
- **Reputation spending**: delegation costs the delegator half of what was granted (cost $= \frac{\text{granted}}{2}$). This bounds the total delegatable reputation to $2 \times$ the original (geometric series), preventing unbounded Sybil creation. The spent reputation is *burned* (not recoverable), creating skin-in-the-game: delegators must exercise due diligence because delegating to a node that later gets slashed permanently reduces their own epistemic capital.
- Equivocation-slashed nodes (reputation = 0) cannot delegate
- Creates a cryptographic trust chain rooted in anchor nodes
- **Rehabilitation:** Slashing is tied to the Ed25519 public key, not to a higher-level identity. A compromised-then-patched device can re-enter by rotating to a fresh key pair and requesting delegation from a trusted anchor (standard enterprise PKI practice).
- **Retroactive collapse prevention:** When a node is slashed, its historical claims could retroactively lose trust weight (since $\text{reputation}_A = 0$ in the propagation formula). Materialized Compaction (Section 7) prevents this: once a subgraph is compacted into a Summary, the Summary reflects the trust topology *at that epoch*. Slashing affects only the uncompacted current epoch and future claims. Historical truth is frozen in Summaries.

6.3. Why Permissioned

We adopt explicit delegation (permissioned model) rather than permissionless reputation bootstrapping. This is a deliberate architectural choice, not a temporary limitation:

Proof-of-accuracy is circular. A node could earn reputation by making predictions that are later confirmed. But "confirmed" requires an oracle of truth — which is itself a node with reputation. The mechanism presupposes what it intends to establish.

Proof-of-stake changes the system's nature. Requiring economic deposits transforms an epistemic protocol into an economic one. The incentive structure (maximize return on stake) diverges from the epistemic goal (maximize belief accuracy).

Computed reputation (EigenTrust-style) is not BFT-safe. PageRank-like iteration over the vote graph uses floating-point arithmetic, breaking the determinism guarantee that is L3′s core contribution. Additionally, Sybil clusters that mutually endorse each other inflate each other's computed reputation — the exact attack vector this system is designed to resist.

For networks where participants are known (IoT fleets, enterprise agent deployments, edge computing consortia), the permissioned model is sufficient and avoids these fundamental issues. For open networks, bridging an external identity layer (e.g., decentralized identifiers with verifiable credentials) to the delegation mechanism is a viable extension that does not require changes to L3′s core algorithm.

6.4. Subjective Epistemic Forks

L3 is *mathematically objective* (given the same inputs, all nodes produce identical outputs) but *epistemically subjective* (the outputs depend on which root anchors are trusted). This is not a defect — it is the resolution to the *Galileo Problem*: what happens when a high-reputation majority suppresses a true minority claim?

Because L3 is a pure computation layer strictly decoupled from L2 data availability, a client can locally evaluate the *same* L2 Merkle-DAG using a *different* set of trusted root anchors. If a cartel of high-reputation nodes acts maliciously, honest nodes do not need to hard-fork the L2 network (as in blockchains); they simply change their local L3 trust anchors. The belief plane will instantly and deterministically re-converge to a new epistemic reality. L3 guarantees consensus *conditional on a shared root of trust*, making epistemic plurality a first-class feature of the protocol rather than a failure mode.

7. Materialized Compaction

7.1. The Split-Brain Problem

L2 epoch GC deletes old DAG nodes. If L3′s Knowledge Graph has edges pointing to deleted nodes, it encounters dangling references.

7.2. Solution

Before epoch GC triggers, L3′s SemanticReducer produces Summary claims and re-injects them into L2 as *new mutations*. The Summary becomes ground truth in the current epoch; old nodes can safely be pruned.

Summaries carry:

- `aggregated_logodds`: combined confidence (log-odds sum of unique sources)
- `variance_milli`: integer variance of input confidences (detects disagreement)
- `range_min`, `range_max`: full confidence spread (preserves outlier information)
- `unique_sources`: count of independent evidence origins
- `evidence_source`: Summary's own BLAKE3 hash (unique, prevents dedup collision)

The SemanticReducer is required to be *commutative*, *associative*, and *idempotent* — the same mathematical properties that make CRDTs converge. This ensures that Summaries produced on different nodes from different DAG states are merge-compatible.

Byzantine Summary defense. Since L3 is deterministic, all honest nodes compute the same Summary from the same input claims. A Byzantine node that injects a fraudulent Summary (e.g., omitting negative claims) will produce a Summary whose BLAKE3 hash differs from what honest nodes compute locally. Honest nodes detect this divergence and apply equivocation slashing to the injector.

Write storm prevention. If all N nodes inject the same Summary simultaneously, N identical mutations flood L2. We prevent this via deterministic leaderless tie-breaking: only the node whose Ed25519 public key has the smallest XOR distance to the BLAKE3 hash of the current L2 epoch ID is authorized to inject the Summary. If it fails (crash or partition), the next-closest node injects after a deterministic timeout (one epoch tick). This avoids both write storms and single points of failure, preserving the leaderless nature of the CRDT layer.

Hybrid Edge Resolution (Graceful Epistemic Degradation). When L2 garbage collection prunes raw claims, epistemic edges targeting those claims would become orphaned. L3 implements *Hybrid Edge Resolution*: edges are transmitted as `RawEpistemicEdge` carrying both the target's `ClaimHash` (exact pointer) and `SemanticFingerprint` (semantic identity). During graph construction, edges resolve via exact hash first (precision mode). If the target hash is missing — compacted into a Summary — the edge falls back to fingerprint resolution, automatically routing to the Summary that inherited the target's semantic identity. Edges where both hash and fingerprint are unresolvable (claims from epochs so old that even their Summaries are gone) are silently dropped. This ensures the belief graph survives GC without synchronous coordination, degrading gracefully from per-claim to per-concept topological accuracy.

8. Belief Engine

The `LogOddsBeliefEngine` classifies claims into three categories:

Category	Condition	Meaning
Accepted	final log-odds ≥ 2197	Believed true (~90%)
Rejected	final log-odds ≤ -2197	Believed false (~10%)
Uncertain	between thresholds	Insufficient evidence

Table 4: Belief Engine classification thresholds.

The pipeline:

1. Compute base trust: $\text{reputation}_{\text{author}(i)} \times \text{confidence}_i$ for each claim
2. Pass 1: fixed-point iteration on acyclic subgraph
3. Pass 2: subtract contradictions (capped, single pass)
4. Classify each claim against thresholds

9. Application: Credential Revocation Propagation

9.1. Motivation

Multi-hop delegation chains in agentic systems — user → orchestrator → sub-agent → tool — propagate authorization forward, but revocation propagation is either pull-based (OAuth RFC 7009 [13], ~100 ms per-hop network latency) or coarse-grained (Privacy Pass [14] batch revocation, minutes).

L3 offers a structural advantage over simple CRL broadcasting: revocation is not merely *announced* but *reasoned about*. A Contradicts edge interacts with the trust graph — a revocation from a high-reputation issuer carries more weight than one from an unknown node. The belief engine can distinguish between legitimate revocation and Byzantine revocation attempts, which a plain broadcast cannot.

9.2. L3 as a Revocation Layer

We map credential operations directly to L3 constructs:

Operation	**L3 Construct**	**Data**
Grant credential	`Claim::Observation`	action_scope, valid_until, pubkey
Delegation	`Relation::DerivedFrom` edge	Parent → child credential
Revocation	`Claim` + `Relation::Contradicts`	Points to revoked grant
Scope check	`BeliefState::Accepted`	Credential has positive belief
Trust chain	`ReputationTracker.delegate()`	Web of Trust = OAuth chain

Table 5: Mapping credential operations to L3 epistemic constructs.

9.3. Measured Results

We benchmark revocation computation across mesh sizes. The table reports **L3 computation time only** — the time for one node to process the revocation claim and update its belief state. End-to-end latency additionally includes L2 gossip propagation (under 1 ms per hop in LAN, measured in Paper 1).

Mesh Size	**Grant (µs)**	**Revoke (µs)**
5 nodes	3.0	5.1
10 nodes	4.2	7.8
20 nodes	7.3	14.3
50 nodes	17.6	34.3
100 nodes	20.2	40.3

Table 6: Credential grant and revocation computation time by mesh size.

Protocol	Revocation Latency	Mechanism
L3 (computation)	40 µs/node	Contradicts edge, local computation
L3 (end-to-end est.)	~1–10 ms	Computation + L2 gossip
OAuth RFC 7009	~100 ms/hop (network)	Token introspection polling
OCSP stapling	~3600 s	Certificate revocation cache
Privacy Pass	~minutes	Batch token rotation

Table 7: Revocation latency comparison. L3 computation is local; OAuth/OCSP include network latency. Direct comparison requires adding L2 gossip to L3 computation.

10. Formal Verification

10.1. TLA+ Specification

We extend AIMP's existing TLA+ methodology (101M states explored for L2 convergence in Paper 1) to the epistemic layer. The specification `AimpBeliefConvergence.tla` models:

- **AddClaim:** A claim arrives at a node with initial trust.
- **AddEdge:** An epistemic edge is created between claims.
- **Replicate:** A node receives claims and edges from another (L2 gossip).
- **PropagatePositive:** Pass 1 trust propagation (Supports edges only).
- **ApplyContradiction:** Pass 2 contradiction subtraction with damping cap.
- **Classify:** Derive belief state from trust values.

Three safety invariants are verified:

Property	Statement
BeliefDeterminism	Same claims + edges + trust $\Rightarrow$ identical beliefs on all nodes
TrustBounded	Trust values remain in $[0, 100]$ after all operations
ContradictionSafety	A single contradiction with $\alpha = 0.5$ cannot flip Accepted $\rightarrow$ Rejected

Table 8: TLA+ safety invariants for the epistemic layer.

Model configuration: 3 nodes, 3 claims, 3 edges, damping cap 50%, accept threshold 60, reject threshold 20.

10.2. Exhaustive Bounded Verification

In addition to TLA+, we implement exhaustive bounded model checking directly in Rust, enumerating all possible graph configurations up to $N = 6$ nodes and verifying safety properties on each. This covers orders of magnitude more configurations than TLC-based verification.

Five properties are verified across 199,902 total configurations:

Property	Configurations	Scope
BeliefDeterminism	729	N=5, 3 confidence levels, all chain edge subsets
TrustBounded	12	N=6, extreme values, dense + cyclic graphs
ContradictionSafety	198,869	All trust $\times$ strength $\times$ damping combinations
CycleSafety	280	Cycles 2–8, all strengths and initial trusts
ConvergenceBounded	12	Chains 3–8, trees breadth 2–4 $\times$ depth 2–3

Table 9: Exhaustive bounded verification: properties and coverage.

Bug found and fixed. The exhaustive tests identified a real bug in the trust propagation implementation. The iteration formula was $\vec{t}_{k+1} = \vec{t}_k + A\vec{t}_k$ (unbounded accumulation) instead of the correct $\vec{t}_{k+1} = \vec{t}_0 + A\vec{t}_k$ (fixed-point iteration resetting to base trust). The incorrect formula caused trust values to grow without bound even with back-edge zeroing, because each iteration added the full contribution to the already-inflated trust rather than recomputing from the base. This bug was missed by six rounds of adversarial AI review and 33 hand-written unit tests, but was caught immediately by the exhaustive verifier on a 2-node cycle with high edge strength. All 199,902 configurations pass after the fix. The bug and its fix are documented in the repository commit history.

10.3. Complexity Analysis

Component	Time	Space	Justification
`LogOdds::aggregate(N)`	$O(N)$	$O(1)$	Linear scan, saturating add
`KnowledgeGraph::detect_cycles()`	$O(V+E)$	$O(V)$	Standard DFS
`cyclic_edge_indices()`	$O(V+E)$	$O(V+E)$	DFS + back-edge set
`propagate_trust_full()` Pass 1	$O(D \times E)$	$O(V)$	D iterations, E edges each. $D \leq V$
`propagate_trust_full()` Pass 2	$O(E)$	$O(V)$	Single pass over Contradicts
`ExactMatchReducer::reduce(N)`	$O(N \log N)$	$O(N)$	Sort + dedup + linear scan
`BeliefEngine::compute()`	$O(V + D \times E)$	$O(V+E)$	Base trust + propagation + classify

Table 10: Per-component complexity analysis of the epistemic layer.

11. Performance Evaluation

All benchmarks run on Apple Silicon (M-series) in release mode. Results are reproducible via `./benchmarks/run_epistemic.sh`. All numbers reported are post-fix (after the trust propagation bug identified in Section 10.2 was corrected).

11.1. Micro-benchmarks (Criterion)

Benchmark	Median	What it measures
`logodds_aggregate/10`	1.3 ns	Log-odds sum of 10 evidence items
`logodds_aggregate/100`	5.5 ns	Log-odds sum of 100 items
`logodds_aggregate/1000`	43.3 ns	Log-odds sum of 1,000 items
`fingerprint_compute`	~1 µs	BLAKE3 dual-key fingerprint
`graph_detect_cycles/1000`	35.2 µs	DFS on 1,000-node ring
`trust_propagation/1000_sparse`	153 µs	Two-pass on 1,000-node chain
`reducer_exact_match/1000`	284 µs	Reduce 1,000 claims
`belief_engine/1000`	140 µs	Full pipeline, 1,000 claims

Table 11: Criterion micro-benchmark results (Apple Silicon, release mode).

11.2. Hot-Path Profiling

The belief engine pipeline breaks down as follows (1,000 claims, 100 iterations averaged):

Step	Time (µs)	% of Total
Base trust computation (rep × conf)	10.6	7.8%
Cycle detection (DFS)	43.3	31.6%
Trust propagation (2-pass)	80.8	59.0%
Classification	2.2	1.6%
TOTAL	136.9	100%

Table 12: Hot-path profile of the belief engine pipeline (1,000 claims).

Trust propagation dominates at 59%, followed by cycle detection at 32%. Classification is negligible (1.6%). This mirrors Paper 1′s profile where Ed25519 signing dominated the L2 hot path at 88.5%: in both cases, the cryptographic/algorithmic core dominates and the surrounding bookkeeping is cheap.

11.3. Scalability

Trust propagation latency scales with graph size and density:

Configuration	Latency (µs)	Edges
10 claims, sparse	1.4	9
100 claims, sparse	6.8	99
1,000 claims, sparse	153.3	999
10,000 claims, sparse	1,749.6	9,999
10 claims, dense	2.0	45
50 claims, dense	133.1	1,225
100 claims, dense	345.8	4,950
200 claims, dense	1,730.8	19,900

Table 13: Trust propagation scalability (sparse = chain, dense = complete graph).

The sparse (chain) topology shows sub-linear scaling: 10× more claims yields ~11× more latency, confirming $O(V + E)$ per iteration. Dense graphs grow as $O(V^2)$ due to $E = O(V^2)$ edges.

11.4. Memory Footprint

Structure	Size (bytes)	Notes
`Claim`	256	Includes `Vec<u8>` data payload
`EpistemicEdge`	12	from + to + relation + strength
`LogOdds`	4	Single `i32`
`Reputation`	2	Single `u16`
`BeliefState`	72	Three `Vec<u32>` (accepted/rejected/uncertain)
`KnowledgeGraph`	56	Edges `Vec` + adjacency `FxHashMap`

Table 14: Per-structure memory footprint.

At 256 bytes per claim and 12 bytes per edge, a 10,000-claim graph with 10,000 edges requires ~2.7 MB of heap memory. At 100,000 claims with 100,000 edges, the L3 state fits in ~27 MB — well within the L3 cache of modern processors (typically 32–64 MB). For edge/IoT deployments, replacing the variable-length `data: Vec<u8>` with a fixed-size hash would reduce `Claim` to ~128 bytes (~1.4 MB for 10,000 claims).

Worst-case execution time. On the densest measured configuration (200 claims, 19,900 edges), trust propagation takes 1.73 ms. For the largest sparse configuration (10,000 claims, 9,999 edges), 1.75 ms. The belief engine pipeline is bounded by $O(D \times E)$ where D is the DFS depth (at most V). In the worst case (10,000-node dense graph with ~$50M$ edges), propagation would take on the order of seconds — but such graphs are unrealistic in practice (each edge requires a signed claim from a reputable node).

11.5. SOTA Comparison

We implement minimal versions of Subjective Logic (Jøsang 2016) and Dempster-Shafer Theory (Shafer 1976) in Rust and benchmark identical workloads:

N	L3 (ns)	Subj. Logic (ns)	Dempster-Shafer (ns)
10	1.3	11.4 (8.6×)	17.1 (13.0×)
100	5.5	357.0 (64.3×)	526.9 (94.9×)
1,000	43.3	4,247.9 (98.1×)	6,163.1 (142.4×)

Table 15: Evidence aggregation performance: L3 vs SOTA.

L3′s advantage grows with N because log-odds aggregation is a single linear scan with saturating addition, while Subjective Logic performs $N - 1$ fusion operations involving division and multiplication, and Dempster-Shafer performs $N - 1$ combination operations over the power set.

Property	L3 (AIMP)	Subjective Logic	Dempster-Shafer
Arithmetic	i32 (deterministic)	f64	f64
BFT compatible	Yes	No*	No*
CRDT integration	Native	External	External
Cycle handling	DFS back-edge zeroing	Discount factors	N/A
Sybil resistance	WoT + rep=0 default	None	None
Known pathologies	None	Vacuous fusion	Zadeh paradox
Complexity	$O(V + E)$	$O(N)$ fusion	$O(2^{\lvert\Theta\rvert})$ combination

Table 16: Feature comparison. *IEEE 754 floats may produce different results across architectures, violating BFT requirements.

11.6. Property-Based Testing

Seven algebraic properties are verified with proptest (256 cases each):

Property	What it tests	Cases
LogOdds commutativity	aggregate([a,b]) == aggregate([b,a])	256
LogOdds associativity	aggregate([a,b,c]) == aggregate([aggregate([a,b]),c])	256
LogOdds no-overflow	aggregate of arbitrary i32 values never panics	256
Reducer commutativity	reduce(forward) == reduce(reversed)	256
Reducer idempotency	reduce(A++A).unique_sources == reduce(A).unique_sources	256
Trust determinism	Two runs produce identical results	64
BeliefEngine determinism	Two runs produce identical classification	64

Table 17: Property-based test coverage (proptest).

All properties pass. Proptest regressions are committed to the repository.

11.7. L3 Overhead on L2 (Network Impairment)

We measure L3′s impact on L2 convergence using in-process network simulation with configurable packet loss, mirroring Paper 1′s netem methodology. A 5-node cluster runs L2 CRDT sync and L3 belief propagation simultaneously.

Configuration	L2 Only (ms)	L2+L3 (ms)	Overhead
10 mut/node	0.145	0.174	20%
50 mut/node	0.502	0.881	75%
100 mut/node	0.899	1.845	105%

Table 18: L3 overhead on L2 convergence (5-node cluster, 0% loss).

The overhead grows with claim count because L3 gossip replicates `Claim` structures (256 bytes each) and `EpistemicEdge` records in addition to L2′s DAG nodes. At 100 mutations/node, L3 roughly doubles convergence time. This cost is dominated by the claim serialization and comparison during gossip, not by the trust propagation itself (which takes only 80.8 µs for 1,000 claims).

Packet Loss	L2 Rounds	L3 Rounds	Both Converge?
0%	1	1	Yes
10%	2	2	Yes
30%	2	2	Yes
50%	3	2	Yes
80%	5	8	Yes

Table 19: L3 belief convergence under packet loss (5 nodes, 20 mutations/node).

Key finding: L3 belief convergence tracks L2 CRDT convergence round-for-round. L3 adds computation overhead (20–105% depending on claim count) but requires **zero additional network rounds** in most scenarios. Under 80% packet loss, both L2 and L3 converge within 8 rounds. This confirms the architectural claim: L3 is a pure computation layer that does not degrade L2′s network convergence properties.

12. Security Analysis

The design addresses 13 vulnerability classes identified during development:

Vulnerability	Mitigation
IEEE 754 non-determinism	Log-odds i32 arithmetic (Section 4)
u16 Bayesian underflow	Log-odds: update = addition, no multiplication
L2/L3 GC split-brain	Materialized compaction (Section 7)
Reducer non-convergence	CAI requirement: commutative, associative, idempotent
Cyclic confidence inflation	DFS back-edge zeroing (Section 5.2)
Trust oscillation	Two-pass separation (Section 5.2)
Hardcoded decay	Dynamic edge strength (Section 5.3)
Confidence gaming	Reputation-gated edge weight
Single-actor annihilation	Configurable contradiction damping cap (α)
Sign-inversion exploit	Trust clamping: $\max(0, \text{source_trust})$ before propagation
Sybil amplification	Zero default reputation + Web of Trust (Section 6)
Type confusion (logodds as rep)	Separate reputation lookup in propagation
Summary evidence collision	Unique `evidence_source` per Summary (BLAKE3 of id)
Unbounded trust accumulation	Fixed-point iteration $t_0 + At_k$ (Section 10.2)
L2/L3 desync (OOM)	State-based snapshot processing under backpressure (Section 3)
Byzantine Summary injection	Deterministic verification + equivocation slashing (Section 7)
Reputation laundering via Reducer	Reputation-aware `reduce_with_reputation()` filters zero-rep claims
ConfidenceInterval i32 overflow	i64 intermediate arithmetic in width computation
Damping Sybil (edge-count α)	Trust-weighted damping ratio replaces raw edge count
Death by thousand cuts	Global penalty cap (α% of stabilized target trust)
Micro-Sybil (1-bps flood)	Reputation-weighted log-odds in Reducer (not raw confidence)
Diamond amplification	Markovian flow normalization (stochastic adjacency matrix)

Table 20: Vulnerability classes and mitigations (22 total).

Each vulnerability is covered by at least one unit test (42 total), and where applicable, by property-based tests (7 properties) and exhaustive bounded verification (5 properties, 199,902 configurations).

13. Discussion

13.1. Significance of the BFT-Determinism Gap

Existing belief aggregation frameworks (Subjective Logic, Dempster-Shafer, EigenTrust) were designed for single-machine or centralized settings where floating-point non-determinism is a minor annoyance. In BFT consensus, where all nodes must agree on identical output to make progress, it becomes a correctness violation. L3 is, to our knowledge, the first belief aggregation framework designed specifically for this constraint.

The 98–142× performance advantage over Subjective Logic and Dempster-Shafer is a secondary benefit. The primary contribution is *architectural*: the guarantee of bit-identical results across ARM64, x86_64, RISC-V, and WebAssembly, which no float-based framework can provide.

13.2. Methodology: Exhaustive Testing as Bug Finding

The trust propagation bug found by exhaustive bounded verification (Section 10.2) is noteworthy for what it reveals about validation methodology. The bug survived: (a) six rounds of adversarial review by three AI systems, (b) 33 hand-written unit tests, and (c) 7 property-based tests with 256 random cases each. It was caught in under 20 ms by the exhaustive verifier on a simple 2-node cycle.

This suggests that bounded model checking over the actual implementation — not a separate formal model — has a higher bug-finding yield per engineering-hour than AI review or randomized testing for this class of algorithm. The bugs that survive hand-written tests are precisely those where the test author shares the same mental model as the implementor. Exhaustive enumeration has no mental model to share.

13.3. Limitations of the Comparison

The SOTA comparison (Section 11.5) benchmarks aggregation speed, which favors L3 because log-odds summation is inherently cheaper than tuple fusion or power-set combination. A fairer comparison would also measure *expressiveness*: Subjective Logic's uncertainty parameter u and Dempster-Shafer's belief functions over power sets can represent epistemic states that L3′s scalar log-odds cannot. L3 trades expressiveness for determinism and speed. Whether this trade-off is acceptable depends on the application.

14. Limitations and Future Work

1. **Naive Bayes independence assumption.** Log-odds summation assumes conditional independence of evidence sources (Section 4.4). Correlated sources produce pathological hyper-confidence. Correlation-aware discounting (e.g., mutual-information weighting) is future work.
2. **Network-level Sybil only.** Echo chamber protection detects amplification (same data relayed by N nodes) but not correlated physical sensor failure.
3. **Permissioned trust.** The Web of Trust requires explicit delegation. The reasons are structural (Section 6.3), not implementation-level. Bridging an external identity layer is the most viable path to permissionless operation.
4. **Credential revocation is simulated.** The P4 benchmark measures L3 computation time with simulated gossip. End-to-end revocation latency in a real deployment depends on L2 gossip speed (under 1 ms per hop in LAN, measured in Paper 1). The L3 netem benchmark (Section 11.8) confirms that L3 does not add network rounds beyond L2.
5. **Single-threaded.** All benchmarks are single-threaded. The trust propagation algorithm is inherently sequential (each iteration depends on the previous), but base trust computation and classification are embarrassingly parallel. Multi-threaded performance is not measured.
6. **Reducer limitations.** The ExactMatchReducer groups by semantic fingerprint equality. Threshold-based and embedding-based grouping (with quantized integer embeddings) are future work.
7. **Reputation inflation partially mitigated.** Reputation spending bounds total delegatable reputation to $2 \times$ original (Section 6.2), but a delegator can still create multiple nodes. Full mitigation would require reputation to be a *conserved quantity* across the network.
8. **Epistemic oligarchy.** Early nodes that accumulate high reputation can suppress emerging truths via high-weight Contradicts edges. Dynamic damping (Section 5.5) partially defends: a well-supported new claim has low α, resisting contradiction from any single actor. Quadratic trust scaling (where influence grows as $\sqrt{\text{reputation}}$ rather than linearly), following the Quadratic Voting framework [15], would further flatten the influence curve and is left to future work.
9. **Bounded verification.** The TLA+ model uses 3 nodes, 3 claims, 3 edges. The exhaustive Rust verification covers 199,902 configurations up to N=6 but is still bounded. A mechanized proof (Coq/Lean) of the convergence claim would provide unbounded guarantees.

10. **Expressiveness trade-off partially addressed.** `ConfidenceInterval` (Section 4.3) adds uncertainty width, but L3 still cannot represent Dempster-Shafer's multi-hypothesis belief functions. Applications requiring power-set beliefs would need multiple L3 claims per hypothesis.

11. **Epistemic Horizon (mitigated).** Epoch compaction could orphan topological edges targeting compacted claims. *Hybrid Edge Resolution* addresses this: edges retain exact `ClaimHash` pointers for precision during the active epoch, but automatically fallback to `SemanticFingerprint` resolution when the target is missing (GC'd). Since Summaries inherit the fingerprint of the claims they compact, orphaned edges route seamlessly to the Summary. Topological accuracy degrades gracefully from per-claim to per-concept granularity — exactly the abstraction level that Summaries represent.

12. **Summary Double-Counting under partition (mitigated).** If two nodes independently compact the same claims during a network partition, the Summaries could have different hashes, causing the CRDT to retain both and double-count log-odds. We mitigate this via *Grid-Aligned Epoch Reduction*: the Reducer operates only on fixed-size temporal grids (e.g., every 10,000 Lamport ticks). Two nodes processing the same claims in the same grid produce byte-identical Summaries (same BLAKE3 hash), which the L2 CRDT deduplicates natively. The grid alignment is deterministic (tick / grid_size), requires no coordination, and adds zero memory overhead. Residual risk: claims at grid boundaries may be compacted one epoch late.

13. **Diamond amplification (mitigated).** Markovian flow normalization (Section 5.2) divides trust across outgoing edges, preventing the worst case of $K \times$ amplification. However, the normalization is per-source-node, not per-origin: if multiple independent sources all point to the same target, their contributions still sum. Full origin-tracking (maximum-flow trust) would provide tighter bounds at $O(V^2 \cdot E)$ cost via standard max-flow algorithms (Edmonds-Karp).

14. **Bounded associativity.** Log-odds aggregation is associative within the operational range ($\pm 10^9$ milli-log-odds, representing probabilities far beyond any physical meaning). At the clamping boundary, sequential aggregation may differ from batch aggregation due to intermediate saturation. This is inherent to any bounded-integer arithmetic and does not affect practical workloads where log-odds values are typically in the ± 7000 range (0.1%–99.9%).

15. Conclusion

We presented an Epistemic Layer for Merkle-DAG CRDTs that transforms raw data convergence into *belief convergence*. The system reasons about trust, contradiction, and evidence quality using deterministic integer arithmetic, making it suitable for Byzantine Fault Tolerant consensus in decentralized multi-agent networks.

Quantitative evaluation demonstrates that L3′s log-odds aggregation is 98–142× faster than Subjective Logic and Dempster-Shafer alternatives, while being the only framework that guarantees bit-identical results across architectures — the prerequisite for Eventual Epistemic Convergence. Exhaustive bounded verification over 199,902 graph configurations identified and helped fix a real bug in the trust propagation algorithm, demonstrating the value of implementation-level model checking.

The credential revocation application shows practical utility: 40 µs per-node revocation computation, with end-to-end latency under 10 ms when combined with L2 gossip. L3 adds 20–105% computation overhead to L2 convergence but requires zero additional network rounds.

By standardizing BFT-deterministic belief aggregation, L3 bridges the gap between distributed systems and multi-agent AI. It provides a foundational *truth layer* where AI agents can autonomously negotiate consensus on physical observations, where contradictions are computationally resolved rather than silently accumulated, and where decentralized networks can reason about the validity of their own data.

The implementation is open-source (Rust, MIT license) and all benchmarks are reproducible via `./benchmarks/run_epistemic.sh`.

Acknowledgments

The design benefited from iterative review by Claude (Anthropic), Gemini (Google), and ChatGPT (OpenAI), which identified 13 vulnerability classes subsequently patched and tested. A fourteenth vulnerability (unbounded trust accumulation) was found by exhaustive bounded verification after the review process. This illustrates that adversarial AI review and formal verification are complementary: the former excels at finding semantic design flaws, the latter at finding implementation bugs.

References

Bibliography

[1] M. Shapiro, N. Preguica, C. Baquero, and M. Zawirski, "Conflict-Free Replicated Data Types," in *Proceedings of the 13th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS)*, 2011.

[2] F. Salmi, "AIMP: AI Mesh Protocol—Design and Evaluation of a Serverless Merkle-CRDT Protocol for Edge Agent Synchronization," 2026. [Online]. Available: https://www.researchgate.net/publication/403127328

[3] M. Kleppmann, "Making CRDTs Byzantine Fault Tolerant," in *Proceedings of the 9th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC)*, 2022.

[4] M. Castro and B. Liskov, "Practical Byzantine Fault Tolerance," in *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI)*, 1999.

[5] E. Buchman, "Tendermint: Byzantine Fault Tolerance in the Age of Blockchains," 2016.

[6] V. Buterin and V. Griffith, "Casper the Friendly Finality Gadget," 2017. [Online]. Available: https://arxiv.org/abs/1710.09437

[7] C. Genest and J. V. Zidek, "Combining Probability Distributions: A Critique and an Annotated Bibliography," *Statistical Science*, vol. 1, no. 1, pp. 114–148, 1986.

[8] E. T. Jaynes, *Probability Theory: The Logic of Science*. Cambridge University Press, 2003.

[9] L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank Citation Ranking: Bringing Order to the Web," *Stanford InfoLab Technical Report*, 1999.

[10] A. Jøsang, *Subjective Logic: A Formalism for Reasoning Under Uncertainty*. Springer, 2016.

[11] G. Shafer, *A Mathematical Theory of Evidence*. Princeton University Press, 1976.

[12] S. D. Kamvar, M. T. Schlosser, and H. Garcia-Molina, "The EigenTrust Algorithm for Reputation Management in P2P Networks," in *Proceedings of the 12th International Conference on World Wide Web (WWW)*, 2003.

[13] T. Lodderstedt, S. Dronia, and M. Scurtescu, "OAuth 2.0 Token Revocation," 2013, RFC7009.

[14] A. Davidson, J. Iyengar, and C. A. Wood, "The Privacy Pass Architecture," 2023, RFC9576.

[15] V. Buterin, Z. Hitzig, and E. G. Weyl, "Liberal Radicalism: A Flexible Design for Philanthropic Matching Funds," *Management Science*, vol. 65, no. 11, pp. 5171–5187, 2019.